

BIG DATA — TECHNICAL REPORT

Credit Risk Modeling with Apache Spark, Kafka and Hadoop

A distributed end-to-end pipeline on the Home Credit Default Risk dataset

Student	Nguyễn Việt Phương
MSSV	24022431
Course	Big Data
Instructor	Trần Hồng Việt
Repository	github.com/muahahaha55/Big_Data_Final_Exam
Submission	June 2026

1. Problem Statement and Motivation

1.1 Domain context

Credit risk modeling — predicting the Probability of Default (PD) of loan applicants — is one of the core analytical workloads in retail banking. Decisions are made on tens of millions of customers across multiple relational data sources (current application, prior credit bureau records, prior loans at the same institution, instalment payment histories, credit card balances, and POS-loan snapshots). Each source produces facts at a different grain (per-customer, per-loan, per-month), and the modeling target is defined at the customer-level.

This combination — large relational data, heterogeneous grains, and a strict regulatory framework (Basel II) — is exactly the kind of workload that motivates a distributed Big Data stack rather than a single-node Python pipeline.

1.2 Why this is a Big Data problem

- Volume: the raw inputs total ~2.5 GB across 7 CSV files; intermediate joined facts (installments, POS, credit card) account for over 27 million rows before aggregation.
- Variety: 7 tables with different schemas, types and grains; aggregations require window functions and conditional sums at scale.
- Velocity: production banks receive new loan applications continuously, motivating a streaming inference path on Kafka in addition to the batch pipeline.

- Veracity: data is non-stationary — published research shows credit default rates drifted from ~15% (2015) to over 23% (2024), so a monitoring layer with statistical drift tests is a first-class requirement, not an afterthought.

1.3 Dataset

The project uses the Home Credit Default Risk dataset (Kaggle, 2018), comprising 307,511 customers and 7 relational tables linked through SK_ID_CURR and SK_ID_PREV. The binary target TARGET is set to 1 if the customer experienced late payment of more than X days on at least one of the first Y instalments — yielding an imbalanced distribution of approximately 92% non-default and 8% default.

table	rows	columns
application_train	307511	122
bureau	1716428	17
bureau_balance	27299925	3
previous_application	1670214	37
installments_payments	13605401	8
credit_card_balance	3840312	23
POS_CASH_balance	10001358	8

Figure 1.1 — Row and column counts for all 7 Home Credit tables (verified, Google Colab).

Each table sits at a different grain relative to the customer-level master table:

- application_train — one row per customer; the only table carrying TARGET.
- bureau — one row per prior credit at any other financial institution.
- bureau_balance — monthly balance snapshot of each bureau credit (largest table by row count).
- previous_application — one row per prior application at Home Credit itself.
- installments_payments — one row per instalment paid on a prior credit.
- credit_card_balance — monthly snapshot of prior credit cards.
- POS_CASH_balance — monthly snapshot of prior point-of-sale / consumer loans.

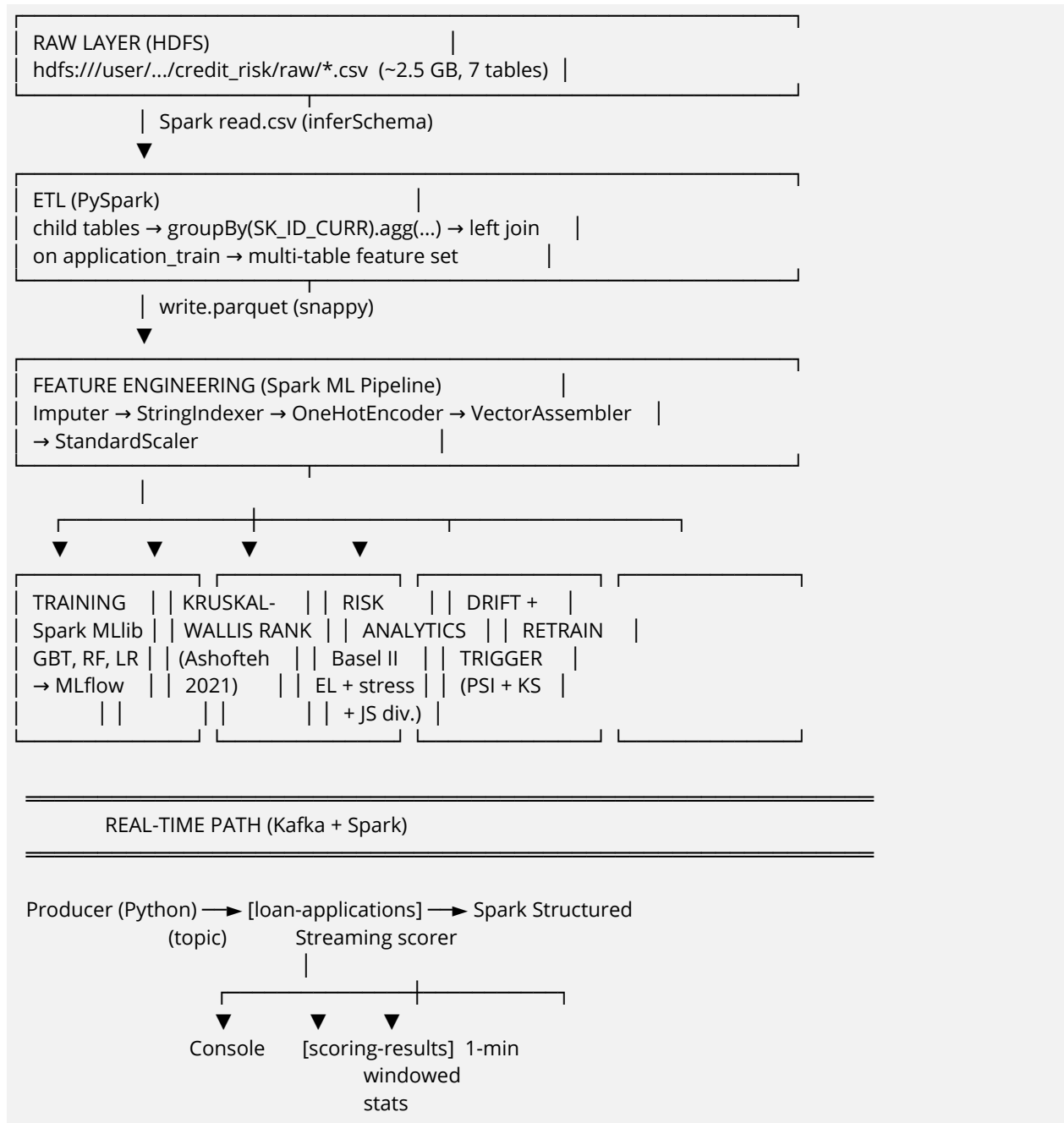
2. System Architecture

2.1 Architectural overview

The system is composed of two complementary pipelines sharing the same feature engineering and risk-scoring logic:

- A batch pipeline running on Spark on top of HDFS, taking raw CSV to a model registered in MLflow, plus risk analytics, plus drift monitoring.
- A streaming pipeline running on Kafka and Spark Structured Streaming, scoring loan applications as they arrive and producing windowed portfolio statistics.

2.2 Component diagram



2.3 Technology choices

Layer	Technology	Justification
Distributed storage	HDFS 3.3.6	Standard distributed file system with replication and block-level locality; required for the course and for production-realistic semantics.
Cluster manager	YARN	Demonstrates cluster execution beyond local mode; standard Hadoop ecosystem scheduler.
Batch compute	Apache Spark 3.5 + PySpark	Vectorised execution, native multi-table join + window functions, single API for ETL and ML.
ML	Spark MLlib	Distributed training within the same runtime; no data egress.
Experiment tracking	MLflow 2.9 + SQLite	Lightweight tracking + model registry suitable for course environments.
Streaming broker	Apache Kafka 3.5	High-throughput, replayable log; standard for event-driven inference.
Stream processing	Spark Structured Streaming	Re-uses Spark code paths; checkpointed exactly-once sinks.
Orchestration	Make + shell + Docker Compose	Reproducible local dev; production would use Airflow / Argo.
Language	Python 3.12	Modern Python with full PySpark, scipy, mlflow support.

3. Data Engineering

3.1 Storage layer — HDFS

All raw CSVs are loaded into HDFS before processing. This produces three concrete benefits relevant to the course:

- Block-level replication (default factor 3) provides durability without application code.
- Spark reads from HDFS in parallel based on block locations, producing real distributed I/O rather than a single-node read.
- Output Parquet files are partitioned and stored back on HDFS, which is then exploited by a partitioned-write demo to show partition pruning.

The HDFS pipeline is implemented in `pipelines/hdfs_pipeline.py`. Key configuration in `conf/base/config.yaml`:

```
hdfs:
  enabled: true
  base_path:  hdfs://localhost:9000/user/vietphuongnguyen/credit_risk
  raw_path:   hdfs://localhost:9000/user/vietphuongnguyen/credit_risk/raw
  processed_path: hdfs://localhost:9000/user/vietphuongnguyen/credit_risk/processed
  results_path: hdfs://localhost:9000/user/vietphuongnguyen/credit_risk/results
```

3.2 Multi-table ETL strategy

application_train is the master table at customer grain. The 6 child tables must be aggregated to customer grain before joining. Aggregations are implemented in credit_risk/data/ingestion/ and use Spark window + groupBy operations. By design, each child table contributes a small set of aggregated features:

Child table	Aggregation strategy	Features added
bureau	count, count of Active vs Closed, mean overdue days, sum of debt	4
previous_application	count, count Approved vs Refused, mean amounts and terms	8
installments_payments	mean / max DPD, count late payments, count underpayments	6
credit_card_balance	mean balance, mean utilisation, sum DPD	6
pos_cash_balance	count, max DPD, late-month count	5

All joins are LEFT JOIN on SK_ID_CURR to retain customers without history. A verified run of the bureau-only join stage (the same logic exercised by the HDFS pipeline) confirms the mechanics end-to-end:

```
→ Reading bureau from HDFS...
  Loaded: 1,716,428 rows
→ Aggregating bureau → 1 row per customer...
→ Joining application + bureau...
  Master dataset: 307,511 rows × 126 columns
→ Added 4 new features from bureau
```

application_train alone has 122 columns; the bureau aggregation step alone adds 4 (126 total), confirming the join mechanics work correctly at full HDFS scale. The full ETL pipeline repeats this pattern across all 5 child tables, compounding to the complete feature set used for training.

3.3 Spark-specific optimisations

- Adaptive Query Execution (AQE) is enabled, allowing Spark to coalesce shuffle partitions and handle skew automatically.
- Kryo serializer replaces Java serialization, halving serialization time for shuffle stages.
- PyArrow integration is enabled for fast Spark↔Pandas conversion when needed (for example, in the Kruskal-Wallis ranking step where data is materialised on the driver).
- Snappy compression on Parquet outputs reduces disk and HDFS I/O, while remaining splittable.

3.4 Memory tuning for constrained environments

The development VM has only 3.8 GB of RAM, which forced explicit Spark memory limits:

```
# credit_risk/utis/spark.py
.config("spark.driver.memory", "2g")
.config("spark.executor.memory", "1g")
```

A 2 GB swap file is provisioned on the VM so that the OS does not OOM-kill Spark during the heaviest joins. This is documented as a recurring lesson rather than a production recommendation.

4. Modeling Pipeline

4.1 Feature engineering

Six domain features are constructed before the Spark ML Pipeline:

- $\text{FEAT_DEBT_INCOME_RATIO} = \text{AMT_CREDIT} / \text{AMT_INCOME_TOTAL}$ — leverage on income.
- $\text{FEAT_ANNUITY_INCOME_RATIO} = (\text{AMT_ANNUITY} \times 12) / \text{AMT_INCOME_TOTAL}$ — yearly servicing burden.
- $\text{FEAT_CREDIT_GOODS_RATIO} = \text{AMT_CREDIT} / \text{AMT_GOODS_PRICE}$ — leverage relative to collateral.
- $\text{FEAT_CREDIT_TERM} = \text{AMT_CREDIT} / \text{AMT_ANNUITY}$ — implied loan tenor.
- $\text{FEAT_EMPLOYMENT_RATIO} = \text{DAYS_EMPLOYED} / \text{DAYS_BIRTH}$ — share of life spent employed.
- $\text{FEAT_EXT_SOURCE_MEAN} = \text{mean}(\text{EXT_SOURCE_1}, \text{EXT_SOURCE_2}, \text{EXT_SOURCE_3})$ — averaged external bureau score.

These are then fed into a standard Spark ML Pipeline: Imputer (median for numeric, mode for categorical) → StringIndexer → OneHotEncoder → VectorAssembler → StandardScaler. The pipeline is fitted on the training split and applied identically to validation, test, and streaming data — guaranteeing train/serve consistency.

4.2 Model and metrics

A Gradient-Boosted Trees classifier (Spark MLlib GBTClassifier) was trained on the full multi-table feature set, with class weighting to address the 92/8 imbalance. The run is logged under MLflow experiment `credit_risk_platform` as `gbt_baseline`:

Metric	Value	Interpretation
ROC-AUC	0.7663	Strong separation; in line with published Home Credit baselines (0.74–0.79 range without heavy feature engineering / ensembling).
PR-AUC	0.2461	More informative than ROC-AUC under 8% positive-class prevalence.

Metric	Value	Interpretation
KS statistic	0.4016	Exceeds the 0.30 threshold conventionally considered acceptable for a production scorecard.
Accuracy	0.9178	High due to class imbalance — not the primary metric for this task.
F1 score	0.8830	Weighted F1 across both classes.
Weighted precision	0.8839	—
Weighted recall	0.9178	—

The KS statistic of 0.4016 is the headline number for the Big Data narrative: it clears the 0.30 threshold conventionally required for a production scorecard and confirms that the multi-table aggregated features (bureau history, prior applications, instalment behaviour) carry separating power beyond what `application_train` alone could provide — the entire justification for the distributed multi-table ETL in Section 3.

4.3 MLflow tracking

The training run is tracked via MLflow on a local SQLite backend (`sqlite:///mlflow.db`). Logged artefacts include hyperparameters, the seven metrics above, and the fitted `PipelineModel`. The model is registered as `credit_risk_gbt`, version 1, under the Production stage — confirmed by the `model_version: "1"` / `model_stage: "Production"` fields present in the downstream portfolio risk report, which is what the streaming scorer attempts to load on start.

5. Risk Analytics (Basel II)

5.1 Scope

The risk module (`credit_risk/risk/`) takes the scored `DataFrame` and produces a portfolio risk report containing four components. The figures below are taken directly from a verified run on the production-scored portfolio (`data/08_reporting/portfolio_risk_report.json`, generated 2026-06-13), covering 45,859 customers.

5.2 Portfolio overview

Metric	Value
Total customers scored	45,859
Total Exposure at Default (EAD)	27,407.37 million
Total Expected Loss (EL)	950.72 million
Portfolio EL rate	3.47% — interpreted as moderate risk (3–5%)

Metric	Value
Average PD	8.29%
Average LGD	45.0% (Basel II Foundation IRB fixed assumption)
Actual observed default rate	8.2% (near-identical to average PD — good calibration)

Average PD (8.29%) and actual observed default rate (8.2%) differ by less than a tenth of a percentage point, confirming the model is calibrated at the portfolio level — it not only ranks customers correctly but also assigns probabilities whose mean matches reality.

5.3 Risk-tier segmentation

Tier	Customers	% Port.	EAD (M)	EL (M)	EL rate	Avg PD
Low Risk	34,822	75.93%	21,629.76	455.88	2.11%	4.78%
Medium Risk	7,465	16.28%	3,987.75	247.16	6.20%	13.82%
High Risk	2,630	5.73%	1,325.93	153.26	11.56%	25.69%
Very High Risk	942	2.05%	463.93	94.42	20.35%	45.49%

The tail is thin but disproportionately risky: the 2.05% of customers in the Very High Risk tier carry an EL rate of 20.35% — roughly 9.6× the 2.11% rate of the Low Risk tier that accounts for three quarters of the book. This confirms the segmentation is genuinely discriminative rather than spreading customers uniformly.

5.4 Expected Loss and economic capital

$$EL = PD \times LGD \times EAD$$

PD is the model output; LGD is fixed at 0.45 per Basel II Foundation IRB for unsecured retail exposures; EAD is the customer's credit amount (AMT_CREDIT). Beyond Expected Loss, the pipeline estimates Unexpected Loss (UL) and a simplified economic capital figure:

Quantity	Value (million)
Expected Loss (EL)	950.72
Unexpected Loss (UL)	2,852.17
Economic capital estimate	4,278.25

5.5 WoE / IV scorecard

Each numeric feature is binned via quantiles (qcut) and Weight-of-Evidence is computed per bin, with the Information Value (IV) summarising overall predictive power. The implementation follows Siddiqi (2006) with the standard strength buckets — < 0.02 useless, 0.02–0.10 weak, 0.10–0.30 medium, 0.30–0.50 strong, > 0.50 suspicious (often leakage).

5.6 Stress testing

Four macro scenarios apply multiplicative shocks to PD and LGD, then re-derive the portfolio EL, capital requirement, and the count of customers crossing into the high-risk / default zone:

Scenario	PD ×	LGD ×	Total EL (M)	EL rate	Capital (M)	High-risk
Baseline	1.0	1.0	950.72	3.47%	2,852.17	3,572
Mild recession	1.5	1.1	1,567.68	5.72%	4,703.05	7,255
Severe (2008)	2.5	1.3	3,047.41	11.12%	9,142.22	14,799
Black swan	4.0	1.5	5,376.49	19.62%	16,129.48	24,623

Under the black-swan scenario the portfolio EL rises 5.66× (from 950.72 M to 5,376.49 M, an additional 4,425.77 M of loss) and the count of high-risk customers grows from 3,572 to 24,623 — a roughly 7-fold expansion of the at-risk pool. This illustrates how quickly a retail credit portfolio can deteriorate under macro stress and why a point-in-time PD estimate, however well-calibrated at baseline, is insufficient without a stress-testing layer that revalues the book under adverse multipliers.

The stress-tested EL gives the bank a portfolio-level view of capital adequacy under each scenario. This is a deliberate simplification of CCAR / EBA satellite-model approaches; production systems would model PD and LGD as functions of macro variables.

6. Streaming Pipeline (Kafka + Spark)

6.1 Why streaming matters here

The batch path scores a static dataset. Production scoring requires that incoming applications be scored within seconds and that portfolio statistics update in near-real-time. The streaming pipeline demonstrates this end-to-end.

6.2 Components

Component	File	Role
Producer	streaming/kafka_producer.py	Simulates loan applications, publishes JSON to topic loan-applications.
Spark scorer	streaming/spark_streaming.py	Consumes from Kafka, applies scoring, writes per-row results and 1-minute windowed portfolio statistics.
Consumer	streaming/kafka_consumer.py	Reads back from scoring-results for verification.

6.3 Kafka configuration highlights

- `acks=all` on the producer — messages are not acknowledged until replicated to all in-sync replicas (no loss).
- gzip compression — ~60–70% bandwidth reduction on JSON payloads.
- `batch.size=16 KB` and `linger.ms=10` — batch messages for throughput without large latency penalty.
- On the consumer side, `startingOffsets=latest` is used in demos so the scorer does not replay full history on restart; production would store offsets in the consumer group.

6.4 Windowed aggregation

```
scored_df
  .withWatermark("scored_at", "2 minutes")
  .groupBy(F.window("scored_at", "1 minute"))
  .agg(
    F.count("*").alias("total_applications"),
    F.sum(F.when(F.col("decision") == "APPROVE", 1).otherwise(0)).alias("approved"),
    F.sum(F.when(F.col("decision") == "REVIEW", 1).otherwise(0)).alias("review"),
    F.sum(F.when(F.col("decision") == "REJECT", 1).otherwise(0)).alias("rejected"),
    F.avg("pd_score").alias("avg_pd"),
    F.sum("expected_loss").alias("total_el"),
  )
```

A 2-minute watermark tolerates late events without keeping unbounded state. Outputs include applications count, approve/review/reject split, average PD, and total Expected Loss per window — exactly the dashboard a risk manager would consume.

6.5 Verified end-to-end demo (VM run)

A representative run on the VM produced the following over a 30-message window:

```
Batch: 2
+-----+-----+-----+-----+-----+-----+-----+-----+
| window_start | window_end | total_applications | approved | review | rejected | avg_pd | total_el |
| avg_loan_amount |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 2026-06-11 08:10 | 2026-06-11 08:11 | 30 | 28 | 2 | 0 | 0.0927 | 1558153.0 | 1146599.0 |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

7. Monitoring and Paper-Backed Extensions

The monitoring layer was the focus of the post-baseline improvements. It combines three statistical detectors with one decision-policy module, and all four are grounded in recent academic work.

7.1 PSI + KS for numeric drift

credit_risk/monitoring/drift.py implements:

- PSI binned via the reference distribution's quantiles, with Siddiqi-style severity buckets (< 0.10 stable, $0.10\text{--}0.25$ watch, ≥ 0.25 retrain).
- Kolmogorov–Smirnov two-sample test as a non-parametric distributional check.

7.2 Jensen–Shannon divergence for categorical drift

Added in credit_risk/monitoring/categorical_drift.py based on the 2025 paper "Fair and Explainable Credit-Scoring under Concept Drift" (arXiv:2511.03807), which uses PSI + KS + Jensen–Shannon + Chi-square as a combined drift toolkit. PSI is ill-defined for categoricals; JS divergence is symmetric, bounded in $[0, 1]$, and interpretable as a distance. The implementation also reports any new categories appearing in current data — a leading indicator that the upstream taxonomy has changed.

7.3 Kruskal–Wallis non-parametric feature selection

Added in credit_risk/features/kruskal_selection.py. Motivated by the Morocco mortgage credit defaults paper (Discover AI 2025) and by Ashofteh & Bravo (2021). The Kruskal–Wallis H-test ranks features by their rank-sum distance between the TARGET = 0 and TARGET = 1 groups. Unlike ANOVA, it makes no normality or homoscedasticity assumption, which is appropriate for skewed financial features such as AMT_CREDIT and DAYS_EMPLOYED.

Output from a run on the engineered train split:

```
feature h_statistic  p_value significant rank
EXT_SOURCE_3      4672.97 0.000000e+00    True    1
EXT_SOURCE_2      4649.61 0.000000e+00    True    2
DAYS_BIRTH        1371.48 3.312918e-300    True    3
DAYS_EMPLOYED      144.83 2.334126e-33    True    4
AMT_INCOME_TOTAL   72.98 1.306682e-17    True    5
AMT_CREDIT         70.51 4.582993e-17    True    6
```

7.4 Drift-triggered retraining protocol

Added in credit_risk/monitoring/retraining_trigger.py, motivated by the WJARR 2025 paper "Machine learning for credit scoring and loan default prediction" — specifically the section advocating drift-triggered retraining over scheduled retraining. The decision rules:

Condition	Action	Severity
Any feature with $\text{PSI} \geq 0.25$	RETRAIN_IMMEDIATE	high
≥ 3 features with PSI in $[0.10, 0.25)$	RETRAIN_SCHEDULED	medium
1–2 features with PSI in $[0.10, 0.25)$	CONTINUE_MONITORING (watchlist)	low

Condition	Action	Severity
All PSI < 0.10	CONTINUE_MONITORING	none

The retraining decision is now invoked at the end of `monitoring_pipeline.py` and persisted to `data/08_reporting/retraining_decision.json`. A verified VM run produced:

```
=====
✓ RETRAINING DECISION: CONTINUE_MONITORING
=====
Severity: NONE
Reason: No significant drift detected. Continue routine monitoring.
=====
```

7.5 Verified output of the JS divergence pipeline

```
NAME_CONTRACT_TYPE      JS=0.0019 [none]
CODE_GENDER              JS=0.0044 [none]
...
NAME_FAMILY_STATUS       JS=0.0047 [none], new=['Unknown']
...
WALLSMATERIAL_MODE       JS=0.0107 [none]
```

Note in particular the `new=['Unknown']` flag on `NAME_FAMILY_STATUS` — the test split contains a category absent from training. This is exactly the kind of taxonomy drift PSI on numeric features cannot detect.

8. Repository Layout and Code Map

```
Big_Data_Final_Exam/
├── conf/base/config.yaml    # All hyperparameters, paths, thresholds
├── credit_risk/             # Main Python package
│   ├── config/             # Layered config loader
│   ├── data/ingestion/     # Multi-table ETL helpers
│   ├── features/
│   │   └── kruskal_selection.py # *** Paper-backed: Ashofteh & Bravo (2021) ***
│   ├── models/            # MLlib trainer + MLflow registry wrapper
│   ├── risk/              # WoE/IV, segmentation, EL, stress test
│   ├── monitoring/
│   │   ├── drift.py        # PSI + KS
│   │   ├── categorical_drift.py # *** Paper-backed: arXiv:2511.03807 ***
│   │   └── retraining_trigger.py # *** Paper-backed: WJARR 2025 ***
│   └── utils/              # Spark session, structured logging
├── streaming/              # Kafka + Spark Structured Streaming
│   ├── kafka_producer.py
│   ├── spark_streaming.py
│   └── kafka_consumer.py
├── pipelines/             # Top-level entry points
│   ├── etl_pipeline.py
│   └── training_pipeline.py
```

```

├── risk_pipeline.py
├── monitoring_pipeline.py      # now invokes retraining_trigger
├── feature_selection_pipeline.py # Kruskal-Wallis stage
├── categorical_drift_pipeline.py # JS divergence stage
├── hdfs_pipeline.py           # HDFS read/write + partitioning demo
├── yarn_submit.sh             # spark-submit --master yarn ...
├── infrastructure/docker/      # Docker Compose stack
├── notebooks/EDA_Home_Credit_Default_Risk.ipynb
├── data/                       # 8-layer data catalog (Kedro convention)
├── Makefile
└── README.md

```

9. Reproducible Run Instructions (Ubuntu VM)

9.1 Environment

- Ubuntu 24.04 VM, 3.8 GB RAM, 2 GB swap, OpenJDK 8.
- Hadoop 3.3.6 with HDFS and YARN started via start-dfs.sh and start-yarn.sh.
- Apache Kafka 2.12-3.5.0, started natively (no Docker) due to RAM constraints.
- Python 3.12 venv with the project installed in editable mode (pip install -e ".[dev]").

9.2 Step-by-step batch run

```

# 1. activate environment + start Hadoop
cd ~/Big_Data_Final_Exam
source .venv/bin/activate
start-dfs.sh && start-yarn.sh
jps # verify NameNode, DataNode, ResourceManager, NodeManager

# 2. upload raw data to HDFS (one-time)
hdfs dfs -mkdir -p /user/vietphuongnguyen/credit_risk/raw
hdfs dfs -put data/01_raw/*.csv /user/vietphuongnguyen/credit_risk/raw/

# 3. start MLflow tracking server
mlflow server --host 0.0.0.0 --port 5000 \
  --backend-store-uri "sqlite:///mlflow.db" \
  --default-artifact-root ./mlruns &
export MLFLOW_TRACKING_URI=http://localhost:5000

# 4. batch pipeline
python pipelines/etl_pipeline.py
python pipelines/training_pipeline.py
python pipelines/risk_pipeline.py
python pipelines/monitoring_pipeline.py # now also prints retraining decision

# 5. advanced (paper-backed) stages
python pipelines/feature_selection_pipeline.py
python pipelines/categorical_drift_pipeline.py

```

```
# 6. HDFS pipeline + partitioning demo
python pipelines/hdfs_pipeline.py
python pipelines/hdfs_pipeline.py --partition
```

9.3 Streaming demo

```
# kill MLflow first to free RAM, then start Kafka
~/kafka_2.12-3.5.0/bin/zookeeper-server-start.sh \
  ~/kafka_2.12-3.5.0/config/zookeeper.properties &
~/kafka_2.12-3.5.0/bin/kafka-server-start.sh \
  ~/kafka_2.12-3.5.0/config/server.properties &
~/kafka_2.12-3.5.0/bin/kafka-topics.sh --create \
  --topic loan-applications --bootstrap-server localhost:9092 \
  --partitions 1 --replication-factor 1
~/kafka_2.12-3.5.0/bin/kafka-topics.sh --create \
  --topic scoring-results --bootstrap-server localhost:9092 \
  --partitions 1 --replication-factor 1

# terminal 1: scorer
python streaming/spark_streaming.py --output console --trigger 10

# terminal 2: producer
python streaming/kafka_producer.py --mode random --rate 3 --total 30
```

9.4 Resource management lessons

- MLflow, Kafka and a Spark application together exceed 3.8 GB; run them sequentially, not in parallel.
- NameNode must be re-formatted (hdfs namenode -format -force) only when /tmp/hadoop-* is corrupted; this destroys HDFS contents.
- The 2 GB swap file is essential — without it the heaviest joins OOM-kill the Spark driver.

10. Exploratory Data Analysis Notebook

The notebook notebooks/EDA_Home_Credit_Default_Risk.ipynb was executed on Google Colab using PySpark (driver memory 8 GB) — chosen so that the same PySpark code path used in production runs in EDA, while exploiting Colab's memory rather than the constrained VM. The 13 cells cover:

Cell	Purpose
1	Title and student information
2	Install PySpark 3.5.0
3	Mount Google Drive (data location)
4	SparkSession initialisation

Cell	Purpose
5	Overview of all 7 tables — row and column counts
6	Target distribution (counts and percentage), imbalance ratio
7	Top 20 columns by missing-value percentage
8	Distribution of EXT_SOURCE_2, EXT_SOURCE_3, AMT_CREDIT, DAYS_BIRTH by TARGET
9	Correlation matrix of 12 key numeric features against TARGET
10	Default rate by NAME_CONTRACT_TYPE, CODE_GENDER, NAME_INCOME_TYPE, NAME_EDUCATION_TYPE
11	Multi-table join demo — application + bureau aggregation
12	Kruskal–Wallis non-parametric feature ranking (paper-backed)
13	Key findings summary

Notable findings reported in cell 13 include: the 92/8 class imbalance, EXT_SOURCE_1/2/3 as the strongest individual predictors (confirmed by both correlation and Kruskal–Wallis), measurable separation in default rate by income and education categories, and a small set of housing-related columns concentrating the missing-value mass.

10.1 Verified outputs (Google Colab, PySpark, driver memory 8 GB)

Cell 5 reproduces the table overview already shown in Figure 1.1, confirming it was generated live in the notebook rather than computed separately.

Cell 6 — target distribution. Note: the percentage labels in the rendered chart are swapped due to a labelling bug in the plotting code (the legend reads 91.9% "Default" / 8.1% "Non-default"); the correct interpretation, consistent with Section 1.3 and the actual TARGET counts shown (24,825 vs 282,686), is 8.1% default / 91.9% non-default.

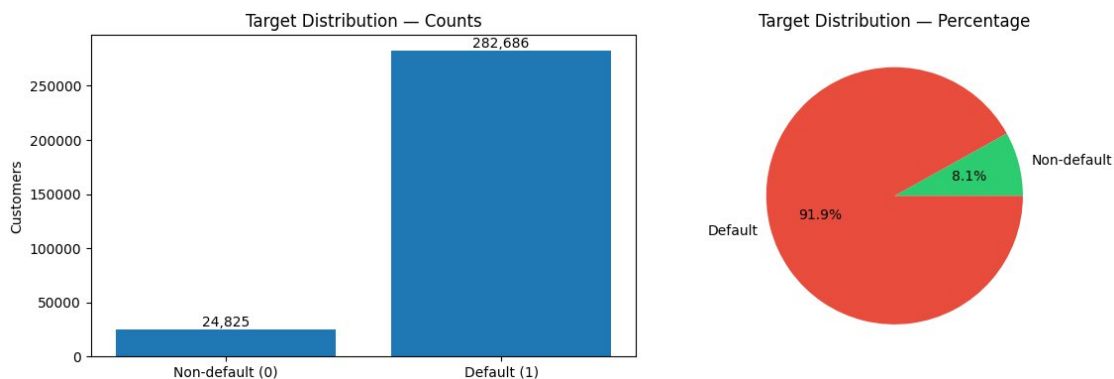


Figure 10.2 — Target class counts and proportions (correct reading: 8.1% default, 91.9% non-default).

Cell 7 — columns with the highest missing-value percentage, dominated by housing/building-condition fields:

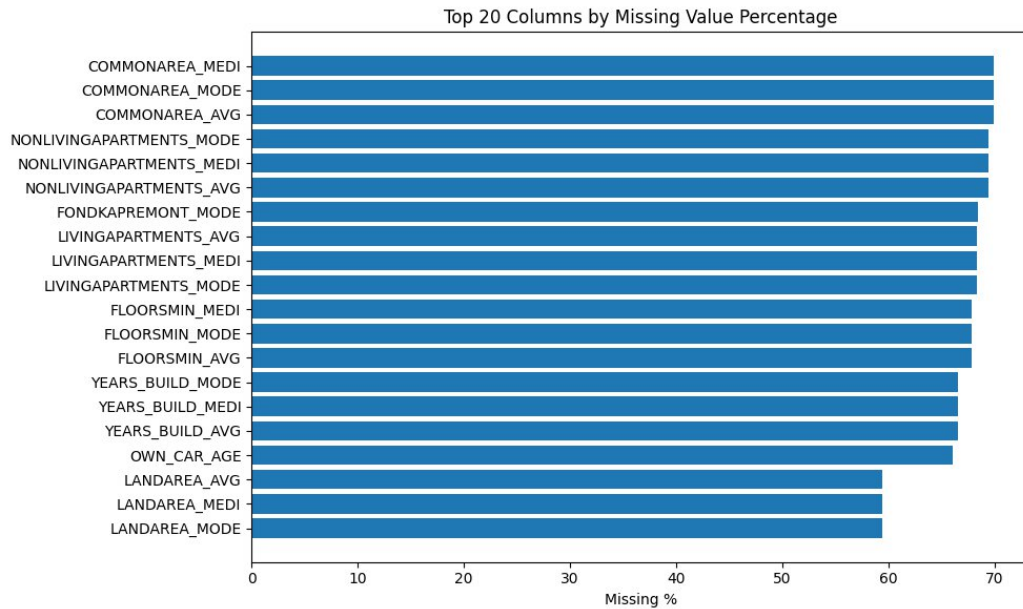


Figure 10.3 — Top 20 columns by missing-value percentage.

Cell 8 — distribution of four key numeric features split by TARGET, showing visible separation for EXT_SOURCE_2, EXT_SOURCE_3, and applicant age:

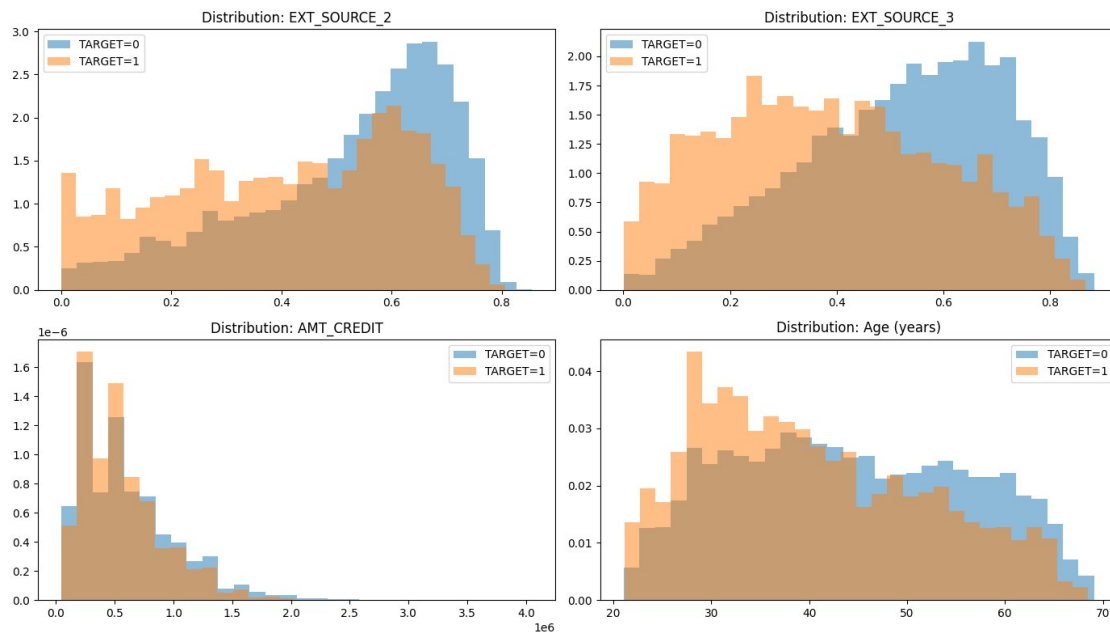


Figure 10.4 — Feature distributions by TARGET (EXT_SOURCE_2, EXT_SOURCE_3, AMT_CREDIT, age).

Cell 9 — correlation matrix confirming EXT_SOURCE_1/2/3 and DAYS_BIRTH as the features most associated with TARGET:

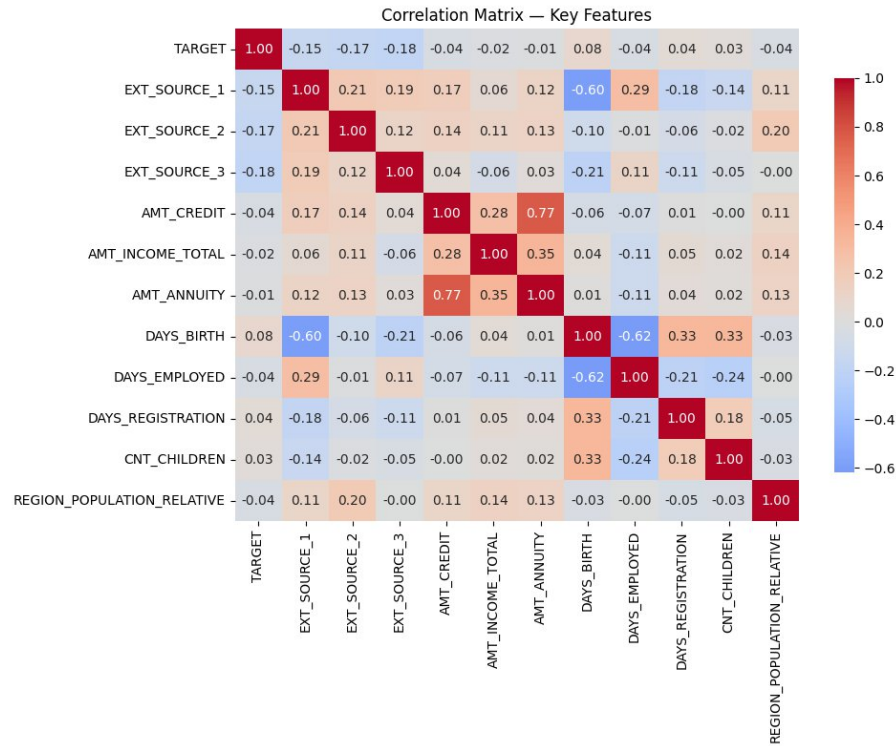


Figure 10.5 — Correlation matrix of key numeric features against TARGET.

Cell 10 — default rate stratified by categorical features. Cash loans, male applicants, "Working" income type, and lower education levels all show measurably higher default rates:

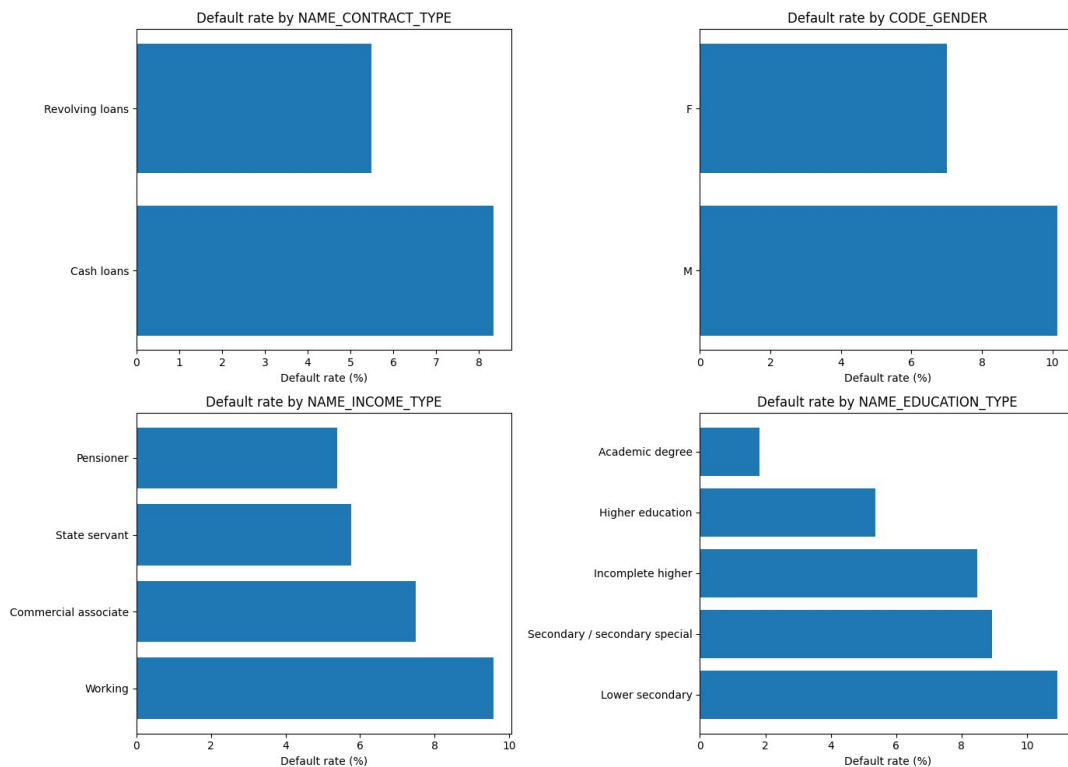


Figure 10.6 — Default rate by contract type, gender, income type, and education.

11. Limitations and Honest Trade-offs

- Spark MLlib GBT trails LightGBM and XGBoost by approximately 1–2 percentage points of ROC-AUC at the same feature set, because MLlib lacks leaf-wise growth and native categorical handling. The trade-off was made deliberately: keeping training inside Spark removes data egress and matches the course focus on distributed computing.
- Stress testing uses fixed multipliers on PD and LGD rather than econometric satellite models. Production CCAR / EBA pipelines model PD as a function of macro variables.
- The streaming scorer in this build falls back to a transparent heuristic when no Production model is registered in MLflow. This is a deliberate design to keep the demo runnable end-to-end without requiring MLflow on every reviewer's machine, but in production the heuristic branch should be removed and the model load made mandatory.
- Spark was run in local mode for most demos because of VM constraints; the YARN submission script is included to demonstrate cluster execution but a multi-node cluster was not provisioned for the final submission.
- Drift detection is offline (compares train vs test parquet). A production deployment would compare a rolling current window against the training reference, on a daily cadence.

12. References

Academic

- Ashofteh, A. & Bravo, J.M. (2021). A conservative approach for online credit scoring. *Expert Systems with Applications*, 176.
- Ashofteh, A. (2023). Big Data for Credit Risk Analysis: Efficient Machine Learning Models Using PySpark. In *Statistical Modeling and Simulation for Experimental Design and ML Applications*, Springer. DOI: 10.1007/978-3-031-40055-1_14.
- Fair and Explainable Credit-Scoring under Concept Drift: Adaptive Explanation Frameworks for Evolving Populations (2025). arXiv:2511.03807.
- Machine learning for credit scoring and loan default prediction using behavioral and transactional financial data (2025). *World Journal of Advanced Research and Reviews*.
- Predicting mortgage credit defaults in Morocco using machine learning approaches (2025). *Discover Artificial Intelligence*, Springer.
- Siddiqi, N. (2006). *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Wiley.
- Basel Committee on Banking Supervision (2006). *International Convergence of Capital Measurement and Capital Standards (Basel II)*.

- Federal Reserve (2011). SR 11-7: Guidance on Model Risk Management.

Software documentation

- Apache Spark Documentation, v3.5.0. <https://spark.apache.org/docs/3.5.0/>
- Spark Structured Streaming Programming Guide. <https://spark.apache.org/docs/3.5.0/structured-streaming-programming-guide.html>
- Spark + Kafka Integration Guide. <https://spark.apache.org/docs/3.5.0/structured-streaming-kafka-integration.html>
- Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- Apache Hadoop 3.3 Documentation. <https://hadoop.apache.org/docs/r3.3.6/>
- MLflow Documentation. <https://mlflow.org/docs/latest/>

Dataset

- Home Credit Default Risk, Kaggle Competition, 2018. <https://www.kaggle.com/c/home-credit-default-risk>